

IF25-11002



Praktikum Pemrograman

Pertemuan 13

Rekursi

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun 2026

```
int faktorial(int n) {  
    if (n <= 1)  
        return 1; // Base case  
    return n * faktorial(n-1);  
}  
  
// faktorial(5) = 120
```

Capaian Pembelajaran

CPL06: Mampu menganalisis dan mengimplementasi konsep dasar matematika, statistika, algoritma dan sistem komputer

Sub-CPMK Pertemuan 13:

- | | | |
|------|--|----|
| 13.1 | Menjelaskan konsep rekursi dan perbedaannya dengan iterasi | C2 |
| 13.2 | Mengidentifikasi base case dan recursive case | C4 |
| 13.3 | Mengimplementasikan function rekursif sederhana | C3 |
| 13.4 | Melakukan tracing eksekusi function rekursif | C4 |
| 13.5 | Mengkonversi solusi iteratif ke rekursif dan sebaliknya | C3 |



Outline Pertemuan 13

150 menit pembelajaran aktif

10'



Review

Recap Function (P12)

15'



Struggle

Tantangan: hitung mundur

50'



Materi

Rekursi: konsep, tracing, contoh

50'



Hands-on

Latihan rekursi bertingkat

25'



Exit Ticket

10 soal assessment



Topik Hari Ini

- Apa itu Rekursi?
- Base Case dan Recursive
- Faktorial Rekursif
- Fibonacci Rekursif
- Tracing Call Stack
- Rekursi vs Iterasi

Analogi Rekursi

Cermin di Dalam Cermin

Matryoshka Doll

Buka boneka → ada boneka

Buka lagi → ada boneka lagi

Buka lagi → ada boneka lagi

...

Sampai boneka TERKECIL!
(tidak bisa dibuka lagi)

= BASE CASE!

Buku dalam Buku

"Untuk memahami rekursi,
Anda harus terlebih dahulu
memahami rekursi."

- Programmer Joke

 Infinite loop tanpa
kondisi berhenti!

 **KUNCI: Rekursi HARUS punya kondisi berhenti (BASE CASE)!**

Tantangan 15 Menit

Misi: Buat Function Hitung Mundur TANPA Loop!

Buat function `hitungMundur(int n)` yang:

- Mencetak `n`, `n-1`, `n-2`, ... sampai 0
- TANPA menggunakan `for/while/do-while`!
- Hint: Function bisa memanggil dirinya sendiri...

Contoh: `hitungMundur(5)`

Output: 5 4 3 2 1 0

Skeleton:

```
void hitungMundur(int n) {  
    // Kapan berhenti?  
    if (???) {  
        return; // STOP!  
    }  
    cout << n << " ";  
    // Panggil diri sendiri?  
    ???  
}
```

 Pikirkan: Kapan harus BERHENTI? Apa yang dilakukan SEBELUM memanggil lagi?

Apa itu Rekursi?



Rekursi = Teknik di mana function **MEMANGGIL DIRINYA SENDIRI** untuk menyelesaikan masalah



Dua Komponen **WAJIB**:



BASE CASE

- Kondisi **BERHENTI**
- Kasus paling sederhana
- **TANPA** pemanggilan rekursif
- Mencegah infinite loop!



RECURSIVE CASE

- Memanggil **DIRI SENDIRI**
- Dengan parameter **BERBEDA**
- Menuju ke base case
- Memecah masalah jadi kecil



TANPA base case = Stack Overflow (program crash karena memori habis)!

Contoh 1: Hitung Mundur

```
void hitungMundur(int n) {  
    // BASE CASE: berhenti jika n < 0  
    if (n < 0) {  
        return; // STOP!  
    }  
  
    // Cetak nilai sekarang  
    cout << n << " ";  
  
    // RECURSIVE CASE: panggil dengan n-1  
    hitungMundur(n - 1);  
}  
  
// hitungMundur(5) → 5 4 3 2 1 0
```

Tracing hitungMundur(3):

```
hitungMundur(3) → cetak 3  
  ↳ hitungMundur(2) → cetak 2  
    ↳ hitungMundur(1) → cetak 1  
      ↳ hitungMundur(0) → cetak 0  
        ↳ hitungMundur(-1)  
          → return (BASE!)
```

Output: 3 2 1 0

Pola Rekursi:

1. Cek BASE CASE dulu (if kondisi_berhenti → return)
2. Lakukan AKSI (cetak, hitung, dll)
3. Panggil DIRI SENDIRI dengan parameter yang MENUJU base case



Contoh 2: Faktorial

IF25-11002 Praktikum Pemrograman

$$n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1 \quad \rightarrow \quad n! = n \times (n-1)!$$

```
int faktorial(int n) {  
    // BASE CASE  
    if (n <= 1) {  
        return 1; // 0! = 1, 1! = 1  
    }  
  
    // RECURSIVE CASE  
    return n * faktorial(n - 1);  
}  
  
// faktorial(5) = 5 * 4 * 3 * 2 * 1 = 120
```

Tracing faktorial(4):

```
faktorial(4)  
= 4 * faktorial(3)  
= 4 * (3 * faktorial(2))  
= 4 * (3 * (2 * faktorial(1)))  
= 4 * (3 * (2 * 1)) ← BASE!  
= 4 * (3 * 2)  
= 4 * 6  
= 24
```

Kunci Faktorial Rekursif:

- Base case: $n \leq 1 \rightarrow$ return 1
- Recursive: return $n * \text{faktorial}(n-1) \rightarrow$ masalah diperkecil setiap pemanggilan



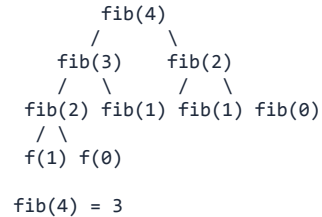
Contoh 3: Fibonacci

IF25-11002 Praktikum Pemrograman

Deret: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... $\rightarrow$ $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

```
int fibonacci(int n) {  
    // BASE CASE  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    // RECURSIVE CASE  
    return fibonacci(n-1) + fibonacci(n-2);  
}  
  
// fibonacci(6) = 8
```

 Tree fib(4):



PERHATIAN: Fibonacci rekursif TIDAK EFISIEN!

- Ada banyak perhitungan BERULANG (fib(2) dihitung berkali-kali)
- Kompleksitas: $O(2^n)$ - sangat lambat untuk n besar
- Solusi: Memoization atau gunakan iterasi untuk kasus nyata



Contoh 4: Pangkat (Power)

IF25-11002 Praktikum Pemrograman

$$\text{base}^{\text{exp}} = \text{base} \times \text{base}^{(\text{exp}-1)} \quad \rightarrow \quad 2^4 = 2 \times 2^3 = 2 \times 2 \times 2 \times 2 = 16$$

```
int pangkat(int base, int exp) {  
    // BASE CASE  
    if (exp == 0) {  
        return 1; // x^0 = 1  
    }  
  
    // RECURSIVE CASE  
    return base * pangkat(base, exp - 1);  
}  
  
// pangkat(2, 4) = 16  
// pangkat(3, 3) = 27
```

Tracing pangkat(2, 4):

```
pangkat(2, 4)  
= 2 * pangkat(2, 3)  
= 2 * (2 * pangkat(2, 2))  
= 2 * (2 * (2 * pangkat(2, 1)))  
= 2 * (2 * (2 * (2 * pangkat(2, 0))))  
= 2 * (2 * (2 * (2 * 1))) ← BASE!  
= 2 * (2 * (2 * 2))  
= 2 * (2 * 4)  
= 2 * 8  
= 16
```



Optimasi: Fast Power dengan $O(\log n)$

$$x^n = (x^{(n/2)})^2 \text{ jika } n \text{ genap} \quad | \quad x^n = x \times x^{(n-1)} \text{ jika } n \text{ ganjil}$$



Contoh 5: Sum Array Rekursif

IF25-11002 Praktikum Pemrograman

$\text{sum}(\text{arr}, n) = \text{arr}[n-1] + \text{sum}(\text{arr}, n-1) \rightarrow$ Jumlahkan dari belakang!

```
int sumArray(int arr[], int n) {  
    // BASE CASE: array kosong  
    if (n == 0) {  
        return 0;  
    }  
  
    // RECURSIVE CASE  
    // Elemen terakhir + sum sisanya  
    return arr[n - 1] + sumArray(arr, n - 1);  
}  
  
int main() {  
    int arr[] = {1, 2, 3, 4, 5};  
    cout << sumArray(arr, 5); // Output: 15  
}
```

 Trace arr={1,2,3}:

```
sumArray(arr, 3)  
= arr[2] + sumArray(arr, 2)  
= 3 + sumArray(arr, 2)  
= 3 + (arr[1] + sumArray(arr, 1))  
= 3 + (2 + sumArray(arr, 1))  
= 3 + (2 + (arr[0] + sumArray(arr, 0)))  
= 3 + (2 + (1 + 0)) ← BASE!  
= 3 + (2 + 1)  
= 3 + 3  
= 6
```



Alternatif: `sumArray(arr + 1, n - 1)` untuk proses dari depan



Rekursi vs Iterasi

IF25-11002 Praktikum Pemrograman

ITERASI (Loop):

```
int faktorialIter(int n) {  
    int hasil = 1;  
    for (int i = 2; i <= n; i++) {  
        hasil *= i;  
    }  
    return hasil;  
}  
  
// Lebih cepat, hemat memori  
// Tapi kadang sulit dipahami
```

REKURSI:

```
int faktorialRek(int n) {  
    if (n <= 1) return 1;  
    return n * faktorialRek(n-1);  
}  
  
// Lebih mudah dipahami  
// Tapi lebih lambat dan pakai  
// banyak memori (call stack)
```

Perbandingan:

Kecepatan

✓ Lebih cepat

✗ Lebih lambat

Memori

✓ Hemat ($O(1)$)

✗ Boros ($O(n)$ stack)

Readability

✗ Bisa kompleks

✓ Lebih intuitif

Best for

Loop sederhana

Tree, divide dan conquer



Call Stack

IF25-11002 Praktikum Pemrograman

Setiap pemanggilan rekursif = PUSH ke stack. Return = POP dari stack.

faktorial(4) - PUSH Phase:

↑ TOP

faktorial(4)	4 * ?
faktorial(3)	3 * ?
faktorial(2)	2 * ?
faktorial(1)	return 1

POP Phase (return):

```
faktorial(1) → return 1
  ↓ pop, kirim ke bawah
faktorial(2) → 2 * 1 = 2
  ↓ pop, kirim ke bawah
faktorial(3) → 3 * 2 = 6
  ↓ pop, kirim ke bawah
faktorial(4) → 4 * 6 = 24
  ↓ pop, SELESAI!
```

Hasil akhir: 24



Stack terlalu dalam = Stack Overflow!

Common Errors Rekursi

Tanpa base case

✗ `int f(n) { return n*f(n-1); }`

✓ `if(n<=1) return 1; ...`

Stack Overflow!

Base case salah

✗ `if (n == 0) // tapi n bisa negatif`

✓ `if (n <= 0) atau if (n <= 1)`

Infinite recursion

Tidak menuju base

✗ `return f(n); // n sama terus`

✓ `return f(n-1); // n berkurang`

Infinite loop

Lupa return

✗ `f(n-1); // tanpa return`

✓ `return f(n-1);`

Hasil hilang

Parameter salah

✗ `fib(n) + fib(n)`

✓ `fib(n-1) + fib(n-2)`

Hasil salah



Selalu: Base case WAJIB, parameter harus MENUJU base case!

Latihan Bertingkat (50 menit)

EASY

15'

Rekursi dasar
Hitung mundur, sum

MEDIUM

20'

Faktorial, pangkat
Fibonacci

CHALLENGE

15'

Rekursi dengan array
Palindrome, GCD

Konsep yang dilatih:

- Base case identification
- Recursive case design
- Call stack tracing
- Rekursi dengan return
- Array processing rekursif

Level Easy: Rekursi Dasar (15')

1

Hitung Maju

Buat `hitungMaju(int n)` yang mencetak 1, 2, 3, ..., n secara rekursif (TANPA loop!).

2

Sum 1 sampai N

Buat `sumTo(int n)` yang mengembalikan $1+2+3+\dots+n$ secara rekursif.

3

Cetak Bintang

Buat `cetakBintang(int n)` yang mencetak n buah bintang (*) secara rekursif.



Selesai semua? Lanjut ke Level Medium!

Level Medium: Rekursi Klasik (20')

4

Faktorial dengan Validasi

Buat faktorial(int n) yang mengembalikan n!. Handle $n < 0$ (return -1 atau error).

5

Pangkat (Power)

Buat pangkat(int base, int exp) yang menghitung base^{exp} secara rekursif.

6

Digit Sum

Buat sumDigit(int n) yang menjumlahkan digit. Contoh: $\text{sumDigit}(123) = 1+2+3 = 6$



Selesai semua? Tantang dirimu di Level Challenge!

Level Challenge: Lanjutan (15')

7 GCD (Euclidean)

Buat `gcd(int a, int b)` secara rekursif.

Algoritma Euclidean:

$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$

Base: $\text{gcd}(a, 0) = a$

Contoh: $\text{gcd}(48, 18) = 6$

8 Palindrome Check

Buat `isPalindrome(string s, int left, int right)` yang mengecek apakah string adalah palindrome.

Base: $\text{left} \geq \text{right} \rightarrow \text{true}$

Rek: $s[\text{left}] == s[\text{right}]?$

Contoh: "radar" = true

Super Challenge: Tower of Hanoi

Buat `hanoi(int n, char from, char to, char aux)` yang menampilkan langkah-langkah memindahkan n piringan dari tiang 'from' ke tiang 'to' menggunakan tiang 'aux'.

Contoh: `hanoi(3, 'A', 'C', 'B')` → cetak 7 langkah pemindahan



Assessment Individual (25 menit)



ATURAN KETAT:

- Dikerjakan SENDIRI
- TANPA bantuan AI/ChatGPT
- TANPA browsing internet
- TANPA diskusi dengan teman
- Pelanggaran = nilai 0



Struktur 10 Soal:

5 soal CODING (50%)
Implementasi rekursi

5 soal TRACING (50%)
Trace call stack dan output

Contoh Soal:

Coding:

Buat function rekursif
`countDigit(int n)` yang menghitung
jumlah digit dalam `n`.

Tracing:

Trace output dari:
`faktorial(5)`
Tunjukkan call stack!



Tugas Minggu Ini

IF25-11002 Praktikum Pemrograman

Tugas: Library Rekursi

Buat program dengan kumpulan function REKURSIF:

1. int faktorial(int n) - hitung n!
2. int fibonacci(int n) - deret fibonacci ke-n
3. int pangkat(int base, int exp) - base^{exp}
4. int sumDigit(int n) - jumlah digit angka
5. int countDigit(int n) - hitung jumlah digit
6. int gcd(int a, int b) - GCD Euclidean
7. bool isPalindrome(string s) - cek palindrome
8. void printBinary(int n) - cetak biner dari desimal

Buat menu interaktif + tampilkan trace setiap pemanggilan!



Deadline: Sebelum Pertemuan 14



Submit via Replit Classroom



Semua HARUS rekursif!



Bonus: Bandingkan dengan iterasi



Referensi Belajar

IF25-11002 Praktikum Pemrograman

Buku

- Deitel - C++ How to Program, Chapter 6.18
- Stroustrup - Programming Principles, Chapter 8

Online

- learncpp.com/cpp-tutorial/recursion/
- [geeksforgeeks.org/recursion/](https://www.geeksforgeeks.org/recursion/)
- visualgo.net/en/recursion (visualisasi)

MK Algoritma

- Pertemuan 13: Konsep Rekursi
- Tracing dan kompleksitas rekursi



Minggu depan: Struct (User-Defined Data Type)!



Kapan Pakai Rekursi?

IF25-11002 Praktikum Pemrograman

✓ PAKAI Rekursi

- Struktur data rekursif (tree, linked list, graph)
- Divide and conquer algorithms
- Backtracking (puzzle, maze)
- Masalah yang natural rekursif (faktorial, fibonacci, tower of hanoi)
- Kode lebih bersih dan intuitif

✗ HINDARI Rekursi

- Loop sederhana cukup (iterasi lebih efisien)
- Kedalaman rekursi sangat dalam (risiko stack overflow)
- Performance critical code
- Perhitungan berulang tanpa memoization (fibonacci naif)
- Keterbatasan memori



Contoh Penggunaan di Dunia Nyata:

- File system traversal (ls -R, find)
- HTML/XML parsing (DOM tree)
- Compiler (syntax tree parsing)
- Quick Sort, Merge Sort algorithms
- Game AI (minimax, chess)
- Fractal generation (Mandelbrot)
- Mathematical proofs
- JSON/data structure processing



Ringkasan Pertemuan 13

IF25-11002 Praktikum Pemrograman

Rekursi

Function yang memanggil DIRINYA SENDIRI

Base Case

Kondisi BERHENTI, tanpa pemanggilan rekursif

Recursive Case

Pemanggilan diri dengan parameter menuju base

Call Stack

Tumpukan pemanggilan: PUSH saat call, POP saat return

vs Iterasi

Rekursi lebih intuitif, iterasi lebih efisien

```
// Template Rekursi
return_type fungsi(parameter) {
    if (base_case_condition) return base_value; // BASE CASE - WAJIB!
    return ... fungsi(smaller_parameter);      // RECURSIVE CASE
}
```



Terima Kasih

IF25-11002 Praktikum Pemrograman

Pertemuan 13: Rekursi

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026